

Systèmes à Microcontrôleurs

Architecture, Programmation Temps Réel et Intégration
Robotique avec ROS

Arduino, ESP32 et Raspberry Pi

Plan de Cours

Équipe Pédagogique

Dr OBONO BIYOBO Arnaud
M. NGONO

Enseignants à l'ENSET – Université de Douala

Année Académique 2025–2026

Niveau : GINFO4

*Ce cours intègre des contenus issus des supports pédagogiques de :
P. Bonnin (Systèmes temps réel), P. Blazevic, O. Snoeck, P. Bonnin (ROS) – ISTY/UVSQ,
et K. Meghriche (Microcontrôleurs) – ISTY/UVSQ.*

Table des matières

Introduction Générale	3
1 Architecture des Microcontrôleurs et Panorama des Plateformes Embarquées	4
1.1 Évolution des Microprocesseurs et Microcontrôleurs	4
1.2 Organisation Interne d'un Microcontrôleur	4
1.3 Plateformes de Développement Embarqué – Étude Comparative	4
1.3.1 Arduino Mega 2560	4
1.3.2 ESP32 (Espressif)	4
1.3.3 Raspberry Pi (Modèles 3, 4 et 5)	4
1.3.4 Comparaison et Choix d'une Plateforme	5
1.4 Prise en Main des Environnements de Développement	5
2 Programmation des Entrées/Sorties Numériques et Analogiques	6
2.1 Entrées/Sorties Numériques (GPIO)	6
2.2 Entrées Analogiques et Conversion Analogique-Numérique (ADC)	6
2.3 Sorties Analogiques par Modulation de Largeur d'Impulsion (PWM)	6
2.4 Commande d'Actionneurs	6
2.5 Affichage d'Informations	6
3 Communications Série et Gestion des Interruptions	7
3.1 Liaison Série Asynchrone (UART)	7
3.2 Bus de Communication pour Capteurs : I2C et SPI	7
3.3 Gestion des Interruptions Matérielles	7
4 Introduction aux Systèmes Temps Réel Embarqués	8
4.1 Définitions et Enjeux	8
4.2 Temps Réel sur Microcontrôleur (Arduino et ESP32)	8
4.3 Temps Réel sous Linux Embarqué (Raspberry Pi)	8
5 Intégration Robotique avec ROS (Robot Operating System)	9
5.1 Pourquoi un Middleware en Robotique?	9
5.2 Concepts Fondamentaux de ROS	9
5.3 Utilisation du Raspberry Pi comme Nœud ROS Natif	9
5.4 Interfaçage Arduino/ESP32 avec ROS via <code>rosserial</code>	9
6 Projets et Perspectives	10
6.1 Projet 1 : Station Météo Connectée avec Interface ROS (ESP32)	10
6.2 Projet 2 : Robot Mobile Différentiel avec ROS (Raspberry Pi + Arduino)	10
6.3 Projet 3 : Contrôleur de Servomoteur par Vision (Raspberry Pi + Caméra)	10
6.4 Ouvertures vers des Systèmes Avancés	10
Modalités d'Évaluation	11
Bibliographie et Ressources	11

Introduction Générale

Contexte et Enjeux des Microcontrôleurs

Les microcontrôleurs sont des circuits intégrés complets (processeur, mémoires, périphériques) qui pilotent la majorité des systèmes électroniques modernes. Leur maîtrise est indispensable pour les ingénieurs en génie informatique et systèmes embarqués.

Applications représentatives :

- **Internet des Objets (IoT) et Industrie 4.0** : Capteurs connectés, actionneurs communicants, maintenance prédictive.
- **Robotique mobile et drones** : Stabilisation, asservissement, fusion de données capteurs, navigation autonome.
- **Automobile** : Calculateurs moteur (ECU), systèmes d'aide à la conduite (ADAS), contrôle de confort.
- **Dispositifs médicaux** : Moniteurs portables, prothèses intelligentes, implants actifs.
- **Domotique et bâtiments intelligents** : Gestion d'énergie, sécurité, éclairage connecté.

Objectifs Pédagogiques

À l'issue de ce cours, les étudiants seront capables de :

1. Analyser l'architecture d'un microcontrôleur et choisir une référence adaptée à un cahier des charges.
2. Programmer en langage C sur différentes plateformes (Arduino, ESP32) et en Python/C sur Raspberry Pi pour piloter des entrées/sorties et des interfaces de communication.
3. Appliquer les principes de base des systèmes temps réel sur microcontrôleur et sur système Linux embarqué.
4. Intégrer un microcontrôleur ou un calculateur embarqué dans un environnement robotique ROS (Robot Operating System).
5. Concevoir et réaliser un projet embarqué complet combinant acquisition capteurs, traitement et communication.

1 Architecture des Microcontrôleurs et Panorama des Plateformes Embarquées

1.1 Évolution des Microprocesseurs et Microcontrôleurs

- Du transistor aux systèmes sur puce (SoC) : repères historiques (Intel 4004, loi de Moore).
- Distinction fondamentale entre microprocesseur et microcontrôleur.
- Architectures Von Neumann et Harvard.

1.2 Organisation Interne d'un Microcontrôleur

- Unité centrale (ALU, registres, unité de contrôle).
- Hiérarchie mémoire : Flash, SRAM, EEPROM.
- Périphériques internes : Timers, ADC, interfaces UART, I2C, SPI, GPIO.
- Critères de sélection d'un microcontrôleur.

1.3 Plateformes de Développement Embarqué – Étude Comparative

Cette section présente en détail les trois plateformes principales utilisées dans le cours.

1.3.1 Arduino Mega 2560

- Microcontrôleur ATmega2560 8 bits à 16 MHz.
- 54 E/S numériques, 16 entrées analogiques, 4 UART.
- Idéal pour l'apprentissage des fondamentaux et le prototypage rapide.
- Programmation en C simplifié via l'IDE Arduino.

1.3.2 ESP32 (Espressif)

- Microcontrôleur 32 bits double-cœur Tensilica LX6 jusqu'à 240 MHz.
- Connectivité Wi-Fi 802.11 b/g/n et Bluetooth 4.2/5.0 intégrée.
- Large gamme de périphériques : ADC, DAC, I2C, SPI, CAN, PWM, etc.
- Programmable avec l'IDE Arduino, ESP-IDF, MicroPython ou FreeRTOS.
- Applications phares : objets connectés (IoT), passerelles sans fil, traitement audio léger.

1.3.3 Raspberry Pi (Modèles 3, 4 et 5)

- Nano-ordinateur monocarte sous Linux (Raspberry Pi OS), processeur ARM Cortex-A multicœur.
- GPIO 40 broches compatibles avec de nombreux capteurs et actionneurs.
- Interfaces : USB, Ethernet, Wi-Fi, Bluetooth, HDMI, CSI (caméra), DSI (écran).
- Programmation en Python, C/C++, Node.js, etc.
- Utilisation comme calculateur embarqué avancé : traitement d'images, intelligence artificielle (TensorFlow Lite), serveur ROS.

1.3.4 Comparaison et Choix d'une Plateforme

Critère	Arduino Mega	ESP32	Raspberry Pi 4
Architecture	8 bits AVR	32 bits Xtensa	64 bits ARM Cortex-A72
Fréquence	16 MHz	240 MHz	1.5 GHz
RAM	8 Ko	520 Ko	2-8 Go
Connectivité sans fil	Non	Wi-Fi, Bluetooth	Wi-Fi, Bluetooth
Système d'exploitation	Aucun (bare metal)	FreeRTOS	Linux
Consommation	Très faible	Faible à modérée	Modérée à élevée
Coût	~ 15	~ 8	~ 40

1.4 Prise en Main des Environnements de Développement

- IDE Arduino pour Mega et ESP32 (installation des cartes additionnelles).
- Raspberry Pi : accès distant SSH, éditeur de code (VS Code, Thonny), compilation GCC.

Travaux Pratiques associés :

- Clignotement de LED sur les trois plateformes.
- Connexion Wi-Fi et scan des réseaux avec l'ESP32.
- Découverte de Linux sur Raspberry Pi et commandes de base.

2 Programmation des Entrées/Sorties Numériques et Analogiques

2.1 Entrées/Sorties Numériques (GPIO)

- Arduino : `pinMode`, `digitalWrite`, `digitalRead`.
- ESP32 : identique à Arduino, avec gestion avancée des interruptions sur toutes les broches.
- Raspberry Pi : bibliothèques `RPi.GPIO` (Python) et `wiringPi` (C).
- Gestion des rebonds sur boutons-poussoirs.

2.2 Entrées Analogiques et Conversion Analogique-Numérique (ADC)

- Arduino Mega : ADC 10 bits, `analogRead()`.
- ESP32 : ADC 12 bits sur plusieurs canaux, configurations avancées (atténuation, calibrage).
- Raspberry Pi : pas d'ADC intégré ; utilisation d'un convertisseur externe MCP3008 via SPI.

2.3 Sorties Analogiques par Modulation de Largeur d'Impulsion (PWM)

- Principe du rapport cyclique.
- Arduino : `analogWrite` (8 bits).
- ESP32 : contrôleur PWM LEDC avec résolution et fréquence configurables (jusqu'à 16 canaux).
- Raspberry Pi : PWM matériel (2 canaux) et logiciel (`gpiozero`).
- Application : variation de luminosité de LED, commande de servomoteurs.

2.4 Commande d'Actionneurs

- Moteur à courant continu avec driver L298N ou L293D.
- Moteur pas-à-pas 28BYJ-48 et driver ULN2003.
- Servomoteur : génération de signaux PWM précis.

2.5 Affichage d'Informations

- Afficheur 7 segments, bargraphe à LED, écran LCD alphanumérique (HD44780).
- Écrans OLED (SSD1306) via I2C – particulièrement adaptés à l'ESP32 et au Raspberry Pi.

Travaux Pratiques associés :

- Voltmètre numérique sur afficheur 7 segments (Arduino Mega).
- Lecture d'un potentiomètre et affichage sur OLED avec l'ESP32.
- Contrôle de servomoteur par PWM sur Raspberry Pi.

3 Communications Série et Gestion des Interruptions

3.1 Liaison Série Asynchrone (UART)

- Principes (TX/RX, débit en bauds).
- Utilisation de l'objet `Serial` sur Arduino/ESP32.
- Sur Raspberry Pi : port série via `/dev/ttyAMA0` ou USB.

3.2 Bus de Communication pour Capteurs : I2C et SPI

- I2C : bus 2 fils, adressage, lecture de capteurs (ex. BME280, MPU6050).
- SPI : bus 4 fils, communication haute vitesse.
- Mise en œuvre sur les trois plateformes.

3.3 Gestion des Interruptions Matérielles

- Intérêt des interruptions versus scrutation (*polling*).
- Arduino/ESP32 : `attachInterrupt()`.
- Raspberry Pi : gestion des interruptions GPIO en Python (`RPi.GPIO`) ou C.
- Règles pour les routines de service d'interruption (ISR).

Travaux Pratiques associés :

- Lecture d'un capteur de température/humidité BME280 sur ESP32 (I2C).
- Détection d'un appui bouton par interruption sur Raspberry Pi.
- Communication SPI entre un Raspberry Pi (maître) et un Arduino (esclave).

4 Introduction aux Systèmes Temps Réel Embarqués

4.1 Définitions et Enjeux

- Système temps réel : exactitude logique et temporelle.
- Temps réel strict (*hard*) vs souple (*soft*).
- Latence, gigue (*jitter*), déterminisme.

4.2 Temps Réel sur Microcontrôleur (Arduino et ESP32)

- Limites de la boucle `loop()`.
- Utilisation de `millis()` et `micros()` pour des attentes non bloquantes.
- Programmation par machine à états.
- Introduction à FreeRTOS (intégré nativement à l'ESP-IDF) : tâches, priorités, files de messages, exclusion mutuelle (mutex, sémaphores).

4.3 Temps Réel sous Linux Embarqué (Raspberry Pi)

- Ordonnancement standard de Linux (`SCHED_OTHER`) et politiques temps réel (`SCHED_FIFO`, `SCHED_RR`).
- Threads POSIX et synchronisation (mutex, variables condition).
- Mesure précise du temps avec `clock_gettime()`.
- Latence et patch `PREEMPT_RT` pour un noyau Linux temps réel.

Travaux Pratiques associés :

- Réécriture d'un chenillard en machine à états sur Arduino Mega.
- Création de deux tâches FreeRTOS sur ESP32 avec priorités différentes.
- Création de threads temps réel sur Raspberry Pi et mesure de la gigue avec `cyclicttest`.

5 Intégration Robotique avec ROS (Robot Operating System)

5.1 Pourquoi un Middleware en Robotique ?

- Modularité, découplage entre perception, décision et action.
- ROS comme standard ouvert.
- Évolution vers ROS 2 pour le support natif du temps réel.

5.2 Concepts Fondamentaux de ROS

- Nœuds (*nodes*), Master (*roscore*).
- Topics (publication/souscription), Services, Actions.
- Outils : `rqt_graph`, RViz, Gazebo.

5.3 Utilisation du Raspberry Pi comme Nœud ROS Natif

- Installation de ROS Noetic sur Raspberry Pi OS.
- Écriture de nœuds ROS en Python ou C++.
- Accès direct aux GPIO depuis un nœud ROS (ex. publication de l'état d'un bouton).

5.4 Interfaçage Arduino/ESP32 avec ROS via `rosserial`

- L'Arduino ou l'ESP32 devient un nœud ROS grâce à la bibliothèque `rosserial`.
- Communication série (USB) pour Arduino, Wi-Fi pour ESP32 (`rosserial_arduino` avec socket TCP).
- Publication de données capteurs et souscription à des commandes.

Travaux Pratiques associés :

- Contrôle de la tortue Turtlesim par un potentiomètre connecté à un Arduino.
- Nœud ROS sur Raspberry Pi qui publie la température d'un capteur BME280.
- Commande à distance d'une LED sur ESP32 via un topic ROS (communication Wi-Fi).

6 Projets et Perspectives

6.1 Projet 1 : Station Météo Connectée avec Interface ROS (ESP32)

- **Matériel** : ESP32, capteur BME280, afficheur OLED SSD1306.
- **Réalisation** : Lecture périodique du capteur, affichage local et publication des données (température, humidité, pression) sur des topics ROS via Wi-Fi.
- **Extension** : Commande à distance d'une LED d'alerte via un topic ROS.

6.2 Projet 2 : Robot Mobile Différentiel avec ROS (Raspberry Pi + Arduino)

- **Matériel** : Raspberry Pi 4 (maître ROS), Arduino Mega (contrôle bas niveau), châssis, moteurs, encodeurs.
- **Réalisation** : Asservissement PID des roues sur Arduino, communication série avec le Raspberry Pi, nœud ROS de pilotage et d'odométrie.

6.3 Projet 3 : Contrôleur de Servomoteur par Vision (Raspberry Pi + Caméra)

- **Matériel** : Raspberry Pi 4, caméra USB ou module caméra CSI, servomoteur.
- **Réalisation** : Traitement d'image avec OpenCV pour détecter un objet coloré, asservissement du servomoteur pour suivre l'objet, publication de la position sur un topic ROS.

6.4 Ouvertures vers des Systèmes Avancés

- **Sécurité des systèmes embarqués** : démarrage sécurisé, mises à jour OTA.
- **Edge AI et TinyML** : déploiement de modèles d'IA sur ESP32 (TensorFlow Lite Micro) et Raspberry Pi (TensorFlow Lite).
- **RTOS industriels** : FreeRTOS, Zephyr, RT-Thread.
- **ROS 2 sur microcontrôleur** : introduction à micro-ROS pour ESP32.

Modalités d'Évaluation

- **Contrôle Continu (40%) :**
 - Comptes rendus de travaux pratiques.
 - Évaluation en ligne sur les concepts théoriques.
- **Projet de Fin de Module (60%) :**
 - Réalisation en binôme d'un projet de synthèse (choix parmi les propositions ou sujet libre validé).
 - Rédaction d'un rapport technique et soutenance avec démonstration.

Bibliographie et Ressources

- **Supports pédagogiques de référence :**
 - Bonnin, P. – *Introduction aux Systèmes d'Exploitation Multi-Tâche et Temps Réel*.
 - Blazevic, P., Snoeck, O., Bonnin, P. – *Initiation ROS sur VM et Raspberry Pi*.
 - Meghriche, K. – *Systèmes à microcontrôleurs*.
 - Sujets de TP Arduino Mega 2560 (ADESFA/ISTY).
- **Références complémentaires :**
 - Margolis, M. – *Arduino Cookbook*, O'Reilly.
 - Banzi, M. & Shiloh, M. – *Getting Started with Arduino*, Make.
 - Kolban, N. – *Kolban's Book on ESP32*.
 - Molloy, D. – *Exploring Raspberry Pi*, Wiley.
 - Joseph, L. – *Robot Operating System (ROS) for Absolute Beginners*, Apress.
- **Sites web officiels :**
 - Arduino : <https://www.arduino.cc/>
 - ESP32 (Espressif) : <https://www.espressif.com/>
 - Raspberry Pi : <https://www.raspberrypi.com/>
 - ROS : <http://wiki.ros.org/>
 - FreeRTOS : <https://www.freertos.org/>